

CampusPulse — Complete Available Backend Source

Important note

This PDF contains all backend files currently available for the CampusPulse replacement backend. The deployed website exposes its frontend publicly, but its original frontend and backend source code are not available for extraction. Therefore, this is not a complete copy of the entire deployed application; it is the complete backend source bundle created for the app.

File 1: server.js

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import Database from 'better-sqlite3';
import { z } from 'zod';

const app = express();
const db = new Database(process.env.DATABASE_FILE || 'campuspulse.db');
const PORT = Number(process.env.PORT || 4000);
const JWT_SECRET = process.env.JWT_SECRET || 'replace-this-secret-in-prod';
const ROLES = ['Student', 'Teacher', 'Administrative head', 'Department head'];
const CATEGORIES = ['Water', 'Electrical', 'Cleanliness', 'Facilities', 'Security'];
const STATUSES = ['Open', 'Acknowledged', 'In progress', 'Resolved', 'Closed'];

app.use(cors({ origin: process.env.FRONTEND_ORIGIN?.split(',') || true, credentials: true }));
app.use(express.json({ limit: '1mb' }));

db.pragma('journal_mode = WAL');
db.exec(`
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT NOT NULL UNIQUE COLLATE NOCASE,
  password_hash TEXT NOT NULL,
  full_name TEXT NOT NULL,
  role TEXT NOT NULL,
  register_number TEXT NOT NULL,
  department TEXT NOT NULL,
  course TEXT NOT NULL,
  availability TEXT,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS reports (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  description TEXT NOT NULL,
  category TEXT NOT NULL,
  location TEXT NOT NULL,
  severity TEXT NOT NULL DEFAULT 'Medium',
  status TEXT NOT NULL DEFAULT 'Open',
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);`);
```

```

reporter_id INTEGER NOT NULL,
reporter_name TEXT NOT NULL,
reporter_role TEXT NOT NULL,
reporter_department TEXT NOT NULL,
reporter_register_number TEXT NOT NULL,
reporter_course TEXT NOT NULL,
assigned_worker_id INTEGER,
resolution_note TEXT,
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (reporter_id) REFERENCES users(id),
FOREIGN KEY (assigned_worker_id) REFERENCES users(id)
);
CREATE TABLE IF NOT EXISTS notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  report_id INTEGER,
  message TEXT NOT NULL,
  read INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (report_id) REFERENCES reports(id)
);
`);

const publicUser = (u) => ({
  id: u.id, email: u.email, fullName: u.full_name, role: u.role,
  registerNumber: u.register_number, department: u.department, course: u.course,
  availability: u.availability || null
});

const sign = (u) => jwt.sign({ sub: u.id, role: u.role }, JWT_SECRET, { expiresIn: 3600 });
const userById = db.prepare('SELECT * FROM users WHERE id = ?');

function auth(required = true) {
  return (req, res, next) => {
    const value = req.headers.authorization;
    if (!value?.startsWith('Bearer ')) {
      if (!required) return next();
      return res.status(401).json({ error: 'Authentication required' });
    }
    try {
      const payload = jwt.verify(value.slice(7), JWT_SECRET);
      const user = userById.get(payload.sub);
      if (!user) return res.status(401).json({ error: 'User no longer exists' });
    } catch {
      return res.status(401).json({ error: 'Invalid token' });
    }
    next();
  };
}

```

```

    req.user = user;
    next();
  } catch { res.status(401).json({ error: 'Invalid or expired token' })
  };
}
function allow(...roles) {
  return (req, res, next) => roles.includes(req.user.role)
    ? next() : res.status(403).json({ error: 'This role cannot perform the'
  }
function validate(schema, source = 'body') {
  return (req, res, next) => {
    const result = schema.safeParse(req[source]);
    if (!result.success) return res.status(400).json({ error: 'Validation'
    req[source] = result.data; next();
  };
}

const accountSchema = z.object({
  email: z.string().email(), password: z.string().min(6), role: z.enum(RO
  fullName: z.string().min(2).max(100), registerNumber: z.string().min(1)
  department: z.string().min(1).max(100), course: z.string().min(1).max(1
  availability: z.string().max(200).optional()
});
const loginSchema = z.object({ email: z.string().email(), password: z.str
const reportSchema = z.object({
  title: z.string().min(3).max(160), description: z.string().min(5).max(5
  category: z.enum(CATEGORIES), location: z.string().min(2).max(160),
  severity: z.enum(['Low', 'Medium', 'High', 'Critical']).default('Medium
  reporterName: z.string().optional(), reporterRole: z.string().optional(
  reporterDepartment: z.string().optional(), reporterRegisterNumber: z.st
  reporterCourse: z.string().optional()
});
const updateSchema = z.object({
  status: z.enum(STATUSES).optional(), assignedWorkerId: z.number().int()
  resolutionNote: z.string().max(2000).optional(), severity: z.enum(['Low
}).refine(x => Object.keys(x).length > 0, 'At least one field is required

app.get('/api/health', (_, res) => res.json({ ok: true, service: 'campuspr

app.post('/api/auth/register', validate(accountSchema), (req, res) => {
  const x = req.body;
  try {
    const result = db.prepare(`INSERT INTO users
      (email,password_hash,full_name,role,register_number,department,cours

```

```

VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`)`.run(x.email.toLowerCase(), bcrypt.hashSync(
  x.registerNumber, x.department, x.course, x.availability || null);
const user = userById.get(result.lastInsertRowid);
res.status(201).json({ token: sign(user), user: publicUser(user) });
} catch (e) {
  if (String(e).includes('UNIQUE')) return res.status(409).json({ error: 'Email already exists' });
  res.status(500).json({ error: 'Could not create account' });
}
});

app.post('/api/auth/login', validate(loginSchema), (req, res) => {
  const user = db.prepare('SELECT * FROM users WHERE email = ? COLLATE NOCASE').get(req.body.email);
  if (!user || !bcrypt.compareSync(req.body.password, user.password_hash)) {
    return res.status(401).json({ error: 'Invalid email or password' });
  }
  res.json({ token: sign(user), user: publicUser(user) });
});

app.get('/api/auth/me', auth(), (req, res) => res.json({ user: publicUser(req.user)}));

const reportSelect = `SELECT r.*, u.full_name AS assigned_worker_name
FROM reports r LEFT JOIN users u ON u.id = r.assigned_worker_id`;
app.get('/api/campus/reports', auth(), (req, res) => {
  const rows = db.prepare(`${reportSelect} ORDER BY r.created_at DESC`).all();
  res.json(rows.map(r => ({ ...r, assignedWorkerName: r.assigned_worker_name })));
});

app.post('/api/campus/reports', auth(), validate(reportSchema), (req, res) => {
  const x = req.body, u = req.user;
  const result = db.prepare(`INSERT INTO reports
(title,description,category,location,severity,reporter_id,reporter_name)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`)`.run(x.title, x.description, x.category,
  u.id, u.full_name, u.role, u.department, u.register_number, u.course);
  const report = db.prepare(`${reportSelect} WHERE r.id = ?`).get(result.lastInsertRowid);
  res.status(201).json(report);
});

app.patch('/api/campus/reports/:id', auth(), allow('Administrative head', 'Admin'), (req, res) => {
  const id = Number(req.params.id), old = db.prepare('SELECT * FROM reports WHERE id = ?').get(id);
  if (!old) return res.status(404).json({ error: 'Report not found' });
  const x = req.body;
  if (x.assignedWorkerId !== undefined) {
    const worker = x.assignedWorkerId && db.prepare("SELECT id FROM users WHERE id = ?").get(x.assignedWorkerId);
    if (x.assignedWorkerId && !worker) return res.status(400).json({ error: 'Invalid worker ID' });
  }
  db.prepare(`UPDATE reports SET status=COALESCE(?,status), assigned_worker_id=COALESCE(?,assigned_worker_id),
  resolution_note=COALESCE(?,resolution_note), severity=COALESCE(?,severity)`.run(x.status ?? null, x.assignedWorkerId === null ? null : (x.assignedWorkerId), x.resolution_note ?? null, x.severity ?? null);
  res.json(db.prepare(`${reportSelect} WHERE r.id = ?`).get(id));
});

```

```

    if (x.status || x.assignedWorkerId !== undefined) {
      db.prepare('INSERT INTO notifications (user_id,report_id,message) VALUES (?, ?, ?)')
        .run(old.reporter_id, id, `Report "${old.title}" was updated${x.status ? ' to ' + x.status : ''}`);
    }
    res.json(db.prepare(`SELECT * FROM notifications WHERE r.id = ?`).get(id));
  });
app.get('/api/campus/workers', auth(), allow('Administrative head', 'Department head'), (req, res) => {
  res.json(db.prepare("SELECT id,full_name AS fullName,department,availability FROM workers").get(id));
});
app.get('/api/notifications', auth(), (req, res) => res.json(db.prepare('SELECT * FROM notifications').get(id)));
app.patch('/api/notifications/:id/read', auth(), (req, res) => {
  db.prepare('UPDATE notifications SET read=1 WHERE id=? AND user_id=?').run(id, req.user.id);
  res.json({ ok: true });
});

app.use((err, _req, res, _next) => { console.error(err); res.status(500).send('Server Error'); });
app.listen(PORT, '0.0.0.0', () => console.log(`CampusPulse backend listening on port ${PORT}`));

```

File 2: package.json

```

{
  "name": "campuspulse-backend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "better-sqlite3": "^11.7.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "zod": "^3.24.1"
  }
}

```

File 3: README.md

CampusPulse backend

This is a clean replacement backend for the public Campus Pulse AI frontend

Run locally

```
```bash
cd campuspulse-backend
npm install
JWT_SECRET="use-a-long-random-secret" npm start
```

The server listens on `http://localhost:4000` by default. Set `PORT`, `DATABASE_FILE`, and `FRONTEND_ORIGIN` as needed. SQLite is created automatically at `campuspulse.db`.

## API

Method	Route	Auth	Purpose
GET	/api/health	No	Health check
POST	/api/auth/register	No	Create a campus account
POST	/api/auth/login	No	Return JWT and user profile
GET	/api/auth/me	Bearer JWT	Return current profile
GET	/api/campus/reports	Bearer JWT	List campus reports
POST	/api/campus/reports	Bearer JWT	Submit a report
PATCH	/api/campus/reports/:id	Admin/department head/worker	Update status, severity, assignment, or resolution
GET	/api/campus/workers	Admin/department head	List workers for assignment
GET	/api/notifications	Bearer JWT	List current-user notifications
PATCH	/api/notifications/:id/read	Bearer JWT	Mark a notification read

Send the JWT as `Authorization: Bearer <token>` .

## Register example

```
{
 "email": "student@campus.edu",
 "password": "change-me",
 "fullName": "Asha Rao",
 "role": "Student",
 "registerNumber": "22CS104",
 "department": "Computer Science",
 "course": "B.Tech"
}
```

## Report example

```
{
 "title": "Water dispenser is leaking",
 "description": "The dispenser near Block A has been leaking since morning",
 "category": "Water",
 "location": "Block A, ground floor",
 "severity": "Medium"
}
```

## Connecting the existing frontend

The public build currently calls procedures named `auth.login`, `auth.register`, `campus.reports`, `campus.workers`, `campus.createReport`, and `campus.updateReport`. If you own the frontend source, either replace those tRPC calls with the REST routes above or add a small tRPC adapter that delegates to these handlers. Do not try to edit the compiled production bundle as the long-term integration path.

## Production checklist

Use PostgreSQL/MySQL instead of SQLite for multi-instance deployment, set a strong secret through a secret manager, restrict `FRONTEND_ORIGIN` to the real frontend origin, enable HTTPS, add rate limiting and email verification, validate role changes server-side, and add audit

logging for administrative updates. Passwords are hashed with bcrypt and are never returned in API responses.

```
File 4: `.env.example`
```

```
```dotenv
```

```
PORT=4000
```

```
DATABASE_FILE=campuspulse.db
```

```
JWT_SECRET=replace-with-a-long-random-secret
```

```
FRONTEND_ORIGIN=http://localhost:5173
```